Digital Banking
Reference Implementation

Volume III of the Digital Banking Engineering Series

John Whitton

Version 1.0.0 · 2026-07-17

Implementation snapshot. All code-backed statements and exact excerpts refer to `johnwhitton/digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`. The repository is the primary implementation artifact. This handbook is not production-ready software, certification, legal advice, or evidence that the full future `V3R-ACC-002` implementation gate has passed.

Reader guide and evidence contract

Volume III begins where the architecture and engineering guidance stop. Volume I defines the system responsibilities and invariants. Volume II explains implementation and operational patterns. This volume walks the concrete repository, line by line where a public contract matters and by module where behavior is best understood through source plus tests.

The primary artifact is the `johnwhitton/digital-banking` repository. Every code-backed statement in this edition is pinned to commit `a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`. A later branch, commit, generated graph, or dirty working tree is outside the evidence set. In particular, uncommitted worker code observed during publication inspection is not documented as current capability.

Production boundary. This is non-production reference software. Durable acceptance is not execution, a pending outbox row is not publication, an interface is not an adapter, and a test is not certification. The pinned repository has no execution worker, no signing runtime, no EVM adapter, no Solana adapter, and no complete reconciliation service.

Status vocabulary

Status	Meaning in this volume
Implemented	Executable bounded behavior and its focused tests exist at the pin.
Partially implemented	A coherent slice exists, but a named owning capability or control remains absent.
Interface only	A public contract and contract-level tests exist; no production or local runtime implementation exists.
Planned	A design or accepted ADR exists without executable source.
Intentionally excluded	The absence is a safety or scope control, not an accidental omission.
Unresolved	The current evidence cannot close the governing requirement.

How code appears

Contracts and public interfaces are exact evidence. Each short excerpt names the repository-relative path, contiguous line range, full commit, and status. The `evidence/excerpt-manifest.csv` file records its SHA-256 digest, and the release verifier compares it with `git show` from the pinned object. The implementation uses `SigningDecision`; there is no public type named `SigningResult` at this pin, and this book does not rename it for stylistic symmetry.

Algorithms and workflows are different. Pseudocode illustrates the engineering pattern without pretending to be committed Java, SQL, Solidity, or Rust. These listings are labeled illustrative and must be specialized only after the owning ADR, module, tests, and production-gap gate exist.

Conformance boundary

The publication itself is a completed Phase 2C artifact. The paired implementation is not a complete implementation of the future Volume III RFC. It does not satisfy `V3R-ACC-002`, which requires the ledger, messaging, signer, EVM, Solana, observation, reconciliation, local runtime, and failure catalog as executable evidence. That gap is preserved rather than silently weakening the specification.

Evidence boundary. The implementation status, module map, test catalog, production-gap register, and excerpt manifest are machine-readable release evidence under `volume3/evidence/`. Architecture remains authoritative over code; code cannot retrofit a new rule without the change-control register.

Reading paths

Read Chapters 1–5 for the concrete Java, Spring, and PostgreSQL slice. Chapters 6–7 show where current public ports stop and native execution must begin. Chapters 8–9 explain what can be tested and run today. Chapter 10 turns every absence into a gated evolution sequence. The appendices collect exact-contract provenance, the complete status table, and implementation terminology.

Contents

List of Figures

List of Tables

1. Repository overview

Why this chapter. A reference implementation is easiest to misread at its directory boundaries. This chapter establishes which modules exist, which dependencies point inward, and which familiar-looking future paths are deliberately absent.

Implementation reference. Maven reactor — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; four committed modules and enforceable inward dependency direction; `pom.xml;domain/pom.xml;application/pom.xml;adapters/persistence-postgres/pom.xml;control-plane/pom.xml`; status: **implemented**.

1.1. Why these modules exist

The repository is small on purpose. `domain/` owns plain-Java financial and lifecycle concepts. `application/` owns use cases and provider-neutral ports. `adapters/persistence-postgres/` translates the application repository contract to explicit PostgreSQL behavior. `control-plane/` is the Spring Boot composition and transport boundary. The root Maven reactor pins the Java/Spring build and applies dependency checks.

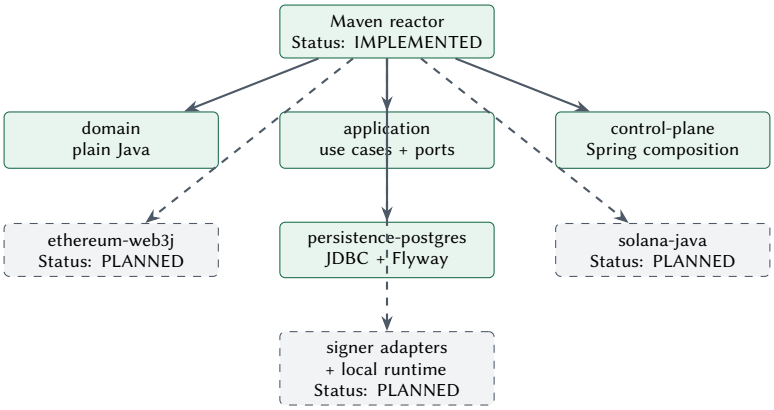

Figure 1.1.. Repository layout at the pinned commit. Solid boxes exist; dashed boxes are approved future paths, not empty modules.

Future names in design documents are not scaffold. There is no `adapters/ethereum-web3j/`, `adapters/solana-java/`, signing-adapter module, `contracts/evm/`, `programs/solana/`, or `integration-tests/` at the evidence commit. Creating empty directories would make the tree appear more complete while proving nothing.

Table 1.1.. Concrete module responsibilities

Module	Owns	Must not own
domain	identity, exact quantity, operation state, attempts, evidence, four finalities	Spring, HTTP, JDBC, JSON, chain SDK, provider types
application	commands, canonicalization, use-case sequencing, ports	web transport, database rows, RPC/native types
persistence-postgres	migration, SQL mapping, transactions, constraints, aggregate reconstruction	business policy, signing, external calls
control-plane	security, request/response mapping, safe errors, configuration, health, composition	domain transition rules or cryptographic authority

1.2. Which decisions the layout implements

Implementation ADR 0001 selects one Maven reactor, Java 25, Spring Boot 4.0.6, Maven Wrapper 3.3.4/Apache Maven 3.9.16, and physical module boundaries. Series ADR-004 places Java in the enterprise control plane; ADR-005 keeps chain-native types behind a language-neutral boundary. Implementation ADR 0004 adds the only current outer adapter: PostgreSQL through explicit JDBC and Flyway. These choices yield the direction in figure 1.2.

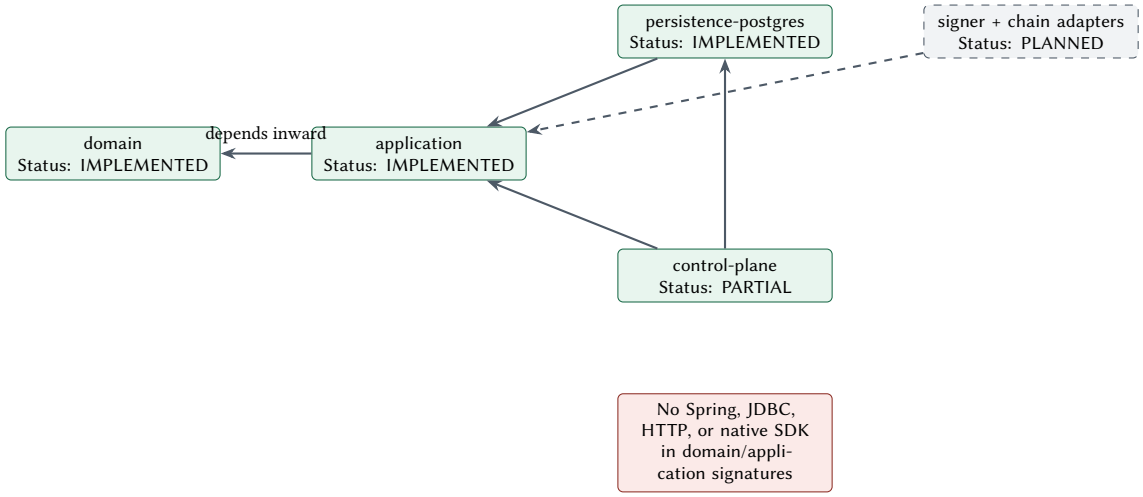

Figure 1.2.. Concrete dependency direction. The prohibition is as important as the arrows.

Dependency direction is a financial-control property, not merely build tidiness. If a receipt, blockhash, SQL row, or Spring annotation enters a domain signature, the core begins to inherit the authority and lifecycle assumptions of an outer system. The Enforcer rules are therefore evidence that prohibited runtime dependencies remain out of the two inner modules; they are not evidence that future adapters are correct.

1.3. Evidence snapshot and repository state

The source tree was inspected read-only. The checkout contained later uncommitted user work, so every source claim and excerpt was resolved with `git show` from the full commit rather than from the filesystem. The committed tree, POMs, tests, ADRs, and living design documents are evidence. `graphify-out/` was used only as a navigation aid and cannot establish behavior.

The implementation repository's committed `docs/reference/` PDFs are immutable historical context and predate the publication repository's current releases. They are not used for page, hash, or current cross-volume claims in this handbook. Current Volume I, Volume II, and Volume III release identity comes from this publication repository; executable behavior comes from the pinned implementation commit.

1.4. How to review a change through the tree

A change should be traced from the authority that requests it, not from the module in which it is easiest to type. For a new operation field, first ask whether it changes the business identity or effect. If it does, the canonical application command and idempotency digest must change. If it constrains legal state, the domain must validate it. If it is merely transport syntax, it stops in the controller mapper. If it is a provider or native-chain detail, it belongs in an outer adapter and reaches the core only through a provider-neutral value or evidence reference.

The reverse trace is equally important. Start at a migration column or response field and follow it inward. Every persisted business field should reconstruct a domain or application concept; every public response field should have an explicit disclosure reason. A database column that no invariant consumes, or a response field copied from a provider object, is a prompt to stop and re-establish ownership. This review method keeps schema and API convenience from becoming accidental architecture.

Table 1.2.. Admission test for a new module

Question	Evidence required	Reject when
Which executable slice owns it?	accepted plan, consumer, module boundary, and exit test	the directory only reserves a future name
Which way may dependencies point?	build rule plus an application-owned contract	provider or framework types enter an inner module
Which authority does it represent?	named decision, state, evidence, and failure owner	two independent authorities are hidden behind one service
What is durable before effects?	transaction and recovery contract	the first durable fact appears after a network call
How is absence reported?	status-register and release limitation update	scaffolding is presented as implementation

At this pin only the PostgreSQL adapter passes this admission test as a concrete outer adapter. The signer and chain ports pass as inner contracts, not as executable modules. That distinction is why the repository can be both useful and deliberately incomplete.

1.5. What remains to be implemented

The repository still needs the execution worker and recovery mechanics before any external effect; the payment/transfer and ledger boundaries; policy and approval implementations; signing service and adapters; EVM and Solana native slices; independent observation; reconciliation and controlled repair; and a complete local runtime. Each new module must have an executable consumer and an accepted owning plan or ADR. The current tree is not production-ready.

Implementation notes. Begin repository review at the root POM, then follow the only allowed direction: control-plane/adapters to application to domain. Treat every absent module in the map as a release limitation, not a contributor TODO that may be created without an owning slice.

Continue the thread. Volume I: architecture boundaries and Java ownership. Volume II: Engineering Foundations and Java/Spring Control Plane. ADRs: series ADR-004/005 and implementation ADR 0001/0004. Shared modules: `shared/engineering/standards/` and `shared/engineering/deployment/`. Implementation paths: root `pom.xml`, module POMs, `docs/DESIGN.md`, and `docs/IMPLEMENTATION.md`.

2. Domain model

Why this chapter. The domain module is where the implementation makes financial and recovery vocabulary impossible to collapse into strings and booleans. Its value lies in the invariants it can enforce without Spring, a database, or a chain.

Implementation reference. Plain-Java domain — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; exact asset/unit quantities, stable operation and attempt identity, guarded token-operation lifecycle, evidence, and four finalities; `domain/src/main/java ;domain/src/test/java`; status: **partially implemented**.

2.1. Why the domain module exists

The domain represents facts the control plane must retain even when every outer system is unavailable: stable operation and attempt identity, exact quantities in a versioned unit, lifecycle state, append-only transition/evidence history, retry authorization, and separate blockchain, legal settlement, customer-visible, and accounting finality. It is deliberately unaware of how these facts arrive over HTTP or are stored in SQL.

Exact pinned excerpt: Operation identity

`digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64 · domain/src/main/java/io/github/johnwhitton/digitalbanking/domain/operation/OperationId .java · lines 7–36`

```
public record OperationId(UUID value) {

    public OperationId {
        Objects.requireNonNull(value, "value");
    }

    public static OperationId from(String canonical) {
        return new OperationId(parseCanonical(canonical));
    }

    @Override
    public String toString() {
        return value.toString();
    }

    private static UUID parseCanonical(String canonical) {
        if (canonical == null) {
            throw new IllegalArgumentException("operation ID is required");
        }
        try {
            UUID parsed = UUID.fromString(canonical);
            if (!parsed.toString().equals(canonical)) {
                throw new IllegalArgumentException("operation ID must use canonical UUID text");
            }
            return parsed;
        } catch (IllegalArgumentException exception) {
            throw new IllegalArgumentException("operation ID must use canonical UUID text", exception);
        }
    }
}
```

This identifier is issued before external work and remains the business correlation across retries. `AttemptId` performs the parallel role for each authorized effort. A native transaction identity, when one eventually exists, is evidence attached to an attempt rather than a replacement for either identifier.

2.2. Exact quantity is necessary but not a ledger

`AssetUnit` carries asset, unit, version, scale, and maximum atomic magnitude. `TokenQuantity` parses canonical decimal text into positive bounded atomic units, rejects excess precision and overflow, and prevents arithmetic across different unit versions. This implements the exact-money portion of the kernel and keeps binary floating point out of value-moving code.

It does not implement double-entry accounting. There are no accounts, posting sets, balanced debits/credits, reservations, correction postings, or accounting-period controls at this pin. PostgreSQL persistence of an exact quantity is durable operation state, not an authoritative ledger. Volume I's ledger responsibility therefore remains an explicit gap.

2.3. Aggregate and lifecycle

`TokenOperation` is the current aggregate. It owns kind (mint or burn), exact quantity, acceptance context, current state/version, ordered attempts, transition history, evidence references, and a history for each of the four finality types. Its tests reject illegal transitions, time regression, duplicate identifiers, blind retry after ambiguity, and finality regression.

The aggregate is broader than a status row but narrower than a payment. It does not own a payment intent, a transfer saga, bank effects, treasury positions, ledger postings, chain observations, reconciliation breaks, or repairs. Completion of a token-operation state machine would still not by itself establish legal or accounting finality.

Table 2.1.. Current domain concepts and authority limits

Concept	Implemented responsibility	Explicit limit
OperationId / AttemptId	stable canonical identities and lineage	not a native chain identifier
AssetUnit / TokenQuantity	versioned scale and exact positive quantity	not price, FX, balance, or ledger
TokenOperation	guarded mint/burn lifecycle and histories	not a full payment or transfer
OperationAttempt	ordered authorized attempts and ambiguity posture	no executable native attempt yet
FinalityRecord	four distinct versioned assessments	no authority/service assesses them yet
EvidenceRef	opaque durable reference	not the evidence payload or telemetry

2.4. Which decisions this implements

ADR-001 makes internal durable state authoritative. ADR-002 preserves four independent finalities. ADR-004 assigns their orchestration to Java, while ADR-005 prevents native chain models from becoming the domain. The use of immutable validated values and defensive copies also implements the repository's Java standards. Domain tests run without Spring, Docker, a database, network access, or a native SDK.

2.5. Reviewing an invariant as code

An invariant review begins with construction. A value that can exist in an invalid state forces every caller to repeat validation and makes deserialization or reconstruction an authority. Constructors and factories should therefore establish canonical form, bounds, and cross-field consistency. The next question is mutation: each transition must name the source states it accepts, the durable facts it adds, and the states it refuses. Finally, reconstruction must travel through the same guards as a new object; persistence is not permission to bypass the model.

For financial state, reviewers should test both the value and its lineage. Equality of two atomic integers is insufficient if their asset, unit version, or scale differs. Likewise, equality of two status names is insufficient if one omits an authorized attempt, an ambiguity decision, or a prior finality assessment. The current model's separate histories make these distinctions expressible, while the missing authorities explain why they are not yet operational conclusions.

Future aggregates should be added at the smallest authority boundary that can own their invariants. A transfer parent may coordinate child bank and token effects, but it should not absorb the ledger. A reconciliation break may refer to an operation and postings, but it should not rewrite either history. A repair command may authorize new evidence or a compensating posting; it must never edit the past into the desired outcome.

Table 2.2.. Domain invariant review worksheet

Review lens	Question	Evidence expected
Identity	Can retry, replacement, and observation preserve the same business lineage?	stable operation plus ordered attempt identifiers
Quantity	Can representation or arithmetic change value silently?	canonical decimal parsing, versioned unit, atomic bounds
Transition	Can a caller skip, regress, or rewrite durable state?	guarded source states and append-only transition record
Ambiguity	Can an uncertain effect become a blind retry?	explicit attempt outcome and inquiry-before-retry rule
Finality	Can one authority impersonate another?	separate histories with named assessment type and version
Evidence	Can a diagnostic string become financial proof?	opaque reference, classification, provenance, and owning store

2.6. What remains to be implemented

Add the payment-intent/transfer parent aggregate only with its owning API and workflow; add an authoritative double-entry ledger with balanced posting invariants; add policy, reconciliation-break, case, and repair aggregates; and connect finality assessment to named authorities and evidence. Native EVM/Solana details must remain outside this module even after adapters exist.

Implementation notes. Use the domain objects to reject impossible state, not to simulate missing outer systems. A test double may supply an application port, but no domain value should pretend that a signature, submission, observation, ledger posting, or settlement occurred.

Continue the thread. Volume I: ledger and state model, payment lifecycle, four finalities. Volume II: durable operation lifecycle, exact money, failure and finality. ADRs: ADR-001/002/004/005. Shared modules: `shared/engineering/standards/` and `shared/engineering/testing/`. Implementation paths: `domain/src/main/java/.../asset/`, `domain/src/main/java/.../operation/`, and matching tests.

3. Application layer

Why this chapter. The application layer turns a validated command into durable business intent while remaining independent of Spring, JDBC, signers, and native chains. It is the seam that prevents transport or provider behavior from becoming workflow authority.

Implementation reference. Application module — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; canonical mint/burn commands, scoped idempotency, durable acceptance orchestration, lookup, and provider-neutral ports; `application/src/main/java;application/src/test/java`; status: **partially implemented**.

3.1. Why this module exists

Controllers should not decide lifecycle, and repositories should not decide business meaning. The application module sits between them. Its command types carry participant scope, operation kind, contract version, asset/unit, exact decimal quantity, and opaque business correlation. `CommandCanonicalizer` produces a versioned SHA-256 digest over effect-relevant fields. `IdempotencyScope` binds tenant, participant, resource, and kind before the repository sees an opaque key.

`TokenOperationApplicationService` validates the configured unit through `AssetUnitCatalog`, creates the canonical command, delegates the single atomic acceptance decision to `OperationRepository`, and performs participant-scoped read-back. `ClockPort` and `IdGenerator` keep time and identities deterministic in tests.

3.2. The persistence contract

Exact pinned excerpt: OperationRepository

`digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/OperationRepository.java · lines 14–32`

```java
/**
 * Persistence contract. Implementations must atomically bind scope/key/canonical metadata
 * to creation.
 */
public interface OperationRepository {

    OperationAcceptance accept(
            IdempotencyScope scope,
            IdempotencyKey key,
            CanonicalCommandMetadata canonicalCommand,
            Supplier<TokenOperation> operationFactory);

    Optional<TokenOperation> findById(OperationId operationId);

    /** Participant-scoped read that must not disclose another participant's operation. */
    Optional<TokenOperation> findById(OperationId operationId, ParticipantScope participant);

    void save(TokenOperation operation, long expectedVersion);
}
```

The key word is "atomically." The application does not call `find` and then `save` to implement acceptance; that would race. The adapter must bind scope, opaque-key digest, canonical metadata, and aggregate creation in one database-authority decision. Same scope/key/digest returns the existing operation. Same scope/key with a different digest is a conflict.

3.3. Current acceptance flow

Listing 3.1. Illustrative pseudocode: current application acceptance

```
validate participant, kind, contract version, configured asset/unit
```

```
canonical = canonicalize(all effect-relevant command fields)
scope = participant + resource + operation kind
return operationRepository.accept(scope, idempotencyKey, canonical, () ->
    TokenOperation.requested(serverOperationId, exactQuantity, acceptanceEvidence))
```

The pseudocode describes committed behavior but is not copied Java. The exact public repository contract above is evidence; orchestration details are read together with `TokenOperationApplicationServiceTest.java` and the PostgreSQL integration tests.

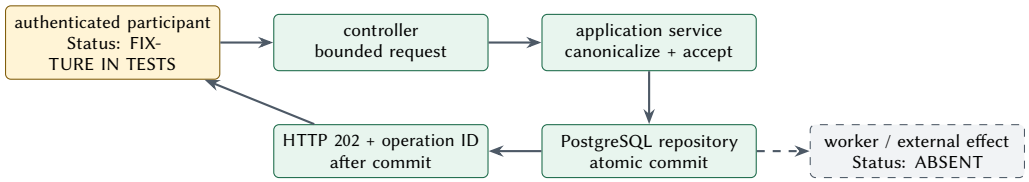

Figure 3.1.. Current request flow. Durable acceptance is implemented; processing is not.

3.4. Ports as authority boundaries

The application owns `OperationRepository`, `AttemptRepository`, `PolicyApprovalPort`, `EvidenceReferencePort`, `SignerPort`, and `ChainPort`. Their location means an adapter implements the application's need; it does not push provider types inward. Only the operation repository has a real outer adapter at the pin. Policy, evidence registration beyond local acceptance, signing, and chain execution remain interface only.

The chain contract deliberately separates prepare, submit once, inquire, and observe. That separation prevents a timeout from becoming a blind second attempt. The signer contract binds exact bytes and effect context. Neither interface authorizes application policy by itself.

3.5. Which decisions this implements

ADR-005 establishes the language-neutral chain boundary. ADR-012 separates signing authority. ADR-013 separates managed submission from independent observation. The application also implements ADR-001's internal truth by returning a durable operation, not a provider acknowledgement. Its runtime dependency is the domain module only.

3.6. Acceptance outcomes and the next command

Durable acceptance has three business outcomes even though database concurrency may take many paths. A first request creates one operation and one future-work intent. An exact replay returns that operation without creating a second intent. A key reused for a different canonical effect is a conflict. Transport retries, thread races, and process restarts must converge on those same outcomes because the repository, rather than an in-memory check, owns the decision.

Table 3.1.. Application interpretation of durable acceptance

Observed condition	Application result	Prohibited inference
New scope/key/digest	return newly accepted operation	worker, policy, signer, or chain has run
Same scope/key/digest	return original operation	a new request or external retry was authorized
Same scope/key, new digest	stable conflict	caller may choose a new key to bypass review
Dependency failure before commit	classified unavailable/error response	acceptance probably happened
Commit succeeds, response is lost	caller replays same scoped request	caller creates a replacement operation

The first future worker command must begin from committed state and a stable event identity. It should acquire database-authoritative ownership for a bounded period, obtain and record required business-policy decisions and approvals, transition the operation to `AUTHORIZED`, then construct and persist the authorized attempt before signing or any external value effect. Every outcome is persisted before the workflow advances. External calls remain outside the acceptance transaction. When a response is lost, the next command is inquiry or observation against the same attempt; it is not "try again" with a fresh native identity.

This sequencing is the practical meaning of the ports. A policy adapter may return an approval decision but cannot create an attempt. A signer may approve exact bytes but cannot choose the operation quantity. A chain adapter may report an ambiguous submit but cannot authorize a replacement. The application owns the workflow transition while each external authority owns only its bounded decision and evidence.

3.7. What remains to be implemented

There is no execution worker, durable timer, lease, inbox, broker publication, policy engine, approval workflow, signing-service client, native adapter orchestration, observation service, reconciliation service, or repair command. Phase 3B must add worker and recovery behavior before any signing or chain slice. A payment/transfer application service remains future work rather than a wrapper around token operations.

Implementation notes. Use the application layer for sequencing and durable intent. Keep every external call outside a database transaction, persist an authorized attempt before effects, and route ambiguous outcomes to inquiry/observation before any new attempt.

Continue the thread. Volume I: component interaction and transaction boundaries. Volume II: Java/Spring control plane, durable workflow, submission/observation. ADRs: ADR-001/005/012/013. Shared modules: `shared/engineering/standards/`, `shared/engineering/custody/`, and `shared/engineering/observability/`. Implementation paths: `application/src/main/java/.../command/`, `application/src/main/java/.../port/`, and application service tests.

4. Spring Boot control plane

Why this chapter. The Spring module is the public and operational edge of the current implementation. Its job is to authenticate, map, classify, and compose—not to hide financial rules in annotations or turn HTTP acceptance into business completion.

Implementation reference. Spring Boot control plane — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; thin mint/burn acceptance and participant-scoped read-back, separate authorities, RFC 9457-style safe errors, health, readiness, and design-first OpenAPI; `control-plane/src/main/java;control-plane/src/main/resources;control-plane/src/test/java`; status: **partially implemented**.

4.1. Why this module exists

`control-plane/` is the composition root. It wires the application service to the PostgreSQL adapter, maps authenticated participants and bounded JSON into framework-free commands, maps durable operations into an allowlisted response, serves the OpenAPI contract, and provides Actuator health/readiness. Spring annotations remain outside the domain and application modules.

Exact pinned excerpt: TokenOperationController endpoints

`digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64 · control-plane/src/main/java/io/github/johnwhitton/digitalbanking/controlplane/api/TokenOperationController.java · lines 39–60`

```
    @PostMapping("/mints")
    public ResponseEntity<TokenOperationResponse> acceptMint(
            @AuthenticationPrincipal ParticipantPrincipal participant,
            @RequestHeader("Idempotency-Key") String idempotencyKey,
            @Valid @RequestBody AcceptanceRequest request) {
        return accept(OperationKind.MINT, participant, idempotencyKey, request);
    }

    @PostMapping("/burns")
    public ResponseEntity<TokenOperationResponse> acceptBurn(
            @AuthenticationPrincipal ParticipantPrincipal participant,
            @RequestHeader("Idempotency-Key") String idempotencyKey,
            @Valid @RequestBody AcceptanceRequest request) {
        return accept(OperationKind.BURN, participant, idempotencyKey, request);
    }

    @GetMapping("/{operationId}")
    public TokenOperationResponse find(
            @AuthenticationPrincipal ParticipantPrincipal participant,
            @PathVariable String operationId) {
        return TokenOperationResponse.from(operations.find(
                parseOperationId(operationId), requireParticipant(participant).scope()));
```

The controller exposes only business resources: accept mint, accept burn, and read an operation within the authenticated participant scope. Callers cannot supply arbitrary calldata, instructions, destinations, contracts, programs, RPC endpoints, key references, or signing requests.

4.2. Commit before HTTP 202

The private controller method delegates acceptance to the application and returns `ResponseEntity.accepted()` only after the repository transaction returns. The response contains a stable `Location` and the durable operation representation. HTTP 202 means the local acceptance transaction committed. It does not mean validation by an execution worker, signing, submission, minting, burning, observation, reconciliation, or settlement.

Exact pinned excerpt: OpenAPI mint contract

`digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64 · control-plane/src/main/resources/static/openapi/token-operations-v1.yaml · lines 9–38`

```
paths:
  /v1/token-operations/mints:
    post:
      operationId: acceptMintOperation
      summary: Durably accept a mint operation request
      description: >-
        A replay with the same participant scope, key, and canonical request
        returns HTTP 202 and the same operation without creating another event.
      security:
        - identityAdapter: []
      x-required-authority: token:mint
      parameters:
        - $ref: '#/components/parameters/IdempotencyKey'
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AcceptanceRequest'
            example:
              contractVersion: 1
              assetId: REFERENCE_ASSET
              unitId: REFERENCE_UNIT
              unitVersion: 3
              quantity: '12.34'
              businessCorrelation: corr-001
      responses:
        '202':
          $ref: '#/components/responses/Accepted'
        '400':
```

The full OpenAPI 3.1 document is the design-first transport contract. It includes mint, burn, participant-scoped read, the contract resource, quantities as decimal strings, separate operation authorities, idempotency-key constraints, safe problem responses, and an accepted-operation example. `OpenApiContractTest.java` checks the file against the implemented surface rather than relying on generated annotations.

4.3. Security and representation boundaries

`SecurityConfiguration` is stateless and deny-by-default. The contract names an identity-adapter boundary, but the default configuration supplies no bearer decoder, issuer, JWK, static token, password, local user, or default principal. Business resources therefore return 401 until an outer identity adapter supplies a `ParticipantPrincipal`. Integration tests inject fixture-only principals and separate `token:mint`, `token:burn`, and `token:read` authorities.

`TokenOperationResponse` is an explicit participant-safe projection. It omits participant scope, idempotency-key digest, canonical request digest, internal evidence, transition actor/reason, and provider details. Only explicitly prefixed participant evidence may cross the response boundary. `ApiExceptionHandler` maps caller, conflict, not-found, dependency, and unexpected application failures to stable, redacted problem types. The filter-chain handlers in `SecurityConfiguration`, not `ApiExceptionHandler`, produce the stable 401/403 responses.

4.4. Which decisions this implements

Series ADR-004 assigns control-plane responsibilities to Java. ADR-001 requires durable internal truth before external representation. Implementation ADR 0001 confines Spring to the outer module, and implementation ADR 0004 makes PostgreSQL mandatory for durable business resources. OpenAPI 3.1 supplies a versioned public API contract rather than an unreviewed framework-generated surface (OpenAPI Initiative 2021).

4.5. Auditing the HTTP boundary

The transport audit follows data in both directions. Inbound, identity and authority come from the security context; the body supplies only bounded business input. The controller maps that input to a framework-free command and delegates once. Outbound, the response is reconstructed from an allowlisted projection rather than serializing an aggregate, exception, database row, or provider object. Error handling classifies the failure while keeping internal identifiers, SQL, stack traces, policy detail, and provider diagnostics out of the participant response.

Table 4.1.. Control-plane boundary audit

Boundary	Required property	Review failure
Principal	participant scope and authority are adapter-derived	body or path can select another participant
Request	bounded fields map to one versioned command	caller supplies native payload, key, route, or provider
Commit	durable repository decision precedes 202	response is sent while acceptance may still roll back
Response	explicit stable projection and location	aggregate or persistence object is serialized directly
Problem	stable type/status with safe detail	raw exception, digest, scope, or dependency detail leaks
Contract	design-first OpenAPI and behavior test agree	annotations create an undocumented second contract

Adding an endpoint therefore requires more than a controller method. The change needs a principal-derived scope, an application use case, durable semantics, idempotency and conflict behavior where effects are requested, a participant-safe representation, problem classification, a design-first contract change, and integration tests covering wrong participant and wrong authority. A management or support endpoint also needs an explicitly different authority and disclosure surface; reusing a participant DTO for operations staff collapses two trust zones.

Readiness must be interpreted with the same restraint. A database-aware readiness check can say whether the current process is prepared for its configured local role. It cannot prove a participant can authenticate, an asset unit is configured, an outbox consumer exists, or settlement dependencies are healthy. Each future dependency should expose a bounded health signal without turning transient provider degradation into false business truth.

4.6. What remains to be implemented

A production identity adapter, issuer trust, authentication mechanism, organization authorization and policy, rate/abuse controls, administrative operations, transfer API, worker endpoints, observation/reconciliation queries, and production operational controls remain absent. The empty default asset catalog also prevents a normal business acceptance outside configured tests. There is no synchronous execution endpoint to add while those durable boundaries are missing.

Implementation notes. Keep controllers thin and participant scope principal-derived. Add a new resource only with a durable use case, transaction/evidence semantics, participant-safe projection, design-first contract, classified errors, and integration tests. Never make a permissive development identity the default.

Continue the thread. Volume I: Java technology ownership, security, and component boundaries. Volume II: Spring API, transaction, security, and failure contracts. ADRs: ADR-001/004 and implementation ADR 0001/0004. Shared modules: `shared/engineering/standards/`, `shared/engineering/compliance/`, and `shared/engineering/testing/`. Implementation paths: `control-plane/src/main/java/.../api/`, `control-plane/src/main/java/.../config/`, and the OpenAPI resource.

5. Persistence

Why this chapter. The current implementation's strongest end-to-end claim is durable acceptance. Understanding it requires following one transaction through schema constraints, idempotency races, aggregate reconstruction, and the pending outbox—and stopping before publication or processing is implied.

Implementation reference. PostgreSQL persistence adapter — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; explicit JDBC, Flyway migration, normalized operation histories, scoped idempotency, concurrency/rollback/restart, and atomic pending outbox; `adapters/persistence-postgres/src/main/java`; `adapters/persistence-postgres/src/main/resources/db/migration`; `adapters/persistence-postgres/src/test/java`; status: **implemented**.

5.1. Why this module exists

The application needs a database-authoritative answer to one question: did this scoped idempotency key and canonical command create a durable operation, replay the same one, or conflict? `PostgresOperationRepository` implements that contract with parameterized explicit SQL, `JdbcClient`, `TransactionTemplate`, and one forward-only Flyway migration. It is verbose because transaction, constraint, and reconstruction behavior must remain reviewable.

Implementation ADR 0004 selects PostgreSQL 17 behavior, pins integration tests to `postgres:17.10-alpine3.23`, rejects JPA/Hibernate and embedded behavioral substitutes, and sets acceptance explicitly to `READ_COMMITTED`. Timestamps are normalized to PostgreSQL microsecond precision before first persistence so accepted and replayed representations remain stable (PostgreSQL Global Development Group 2026; Redgate 2026).

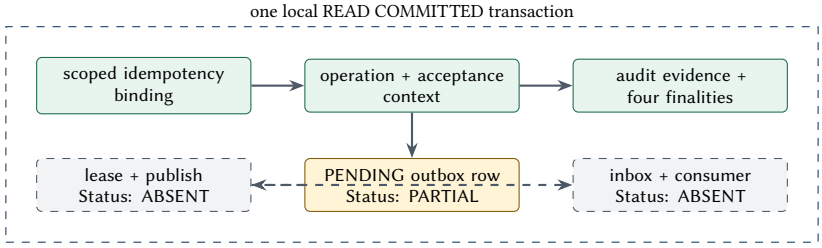

Figure 5.1.. Persistence flow. Atomic intent exists; at-least-once delivery and consumption remain future work.

5.2. What commits together

The migration normalizes operation acceptance context, idempotency binding, operation state/version, transition/evidence history, attempt lineage, four finality histories, and a versioned outbox table. Exact quantities are positive bounded `NUMERIC(512,0)` atomic units plus their versioned asset/unit definition. Database constraints protect important cardinality, kind, state, version, ordering, and uniqueness rules.

1. Hash the opaque idempotency key; never persist or return the raw key.
2. Try the scoped binding with `INSERT ... ON CONFLICT DO NOTHING`.
3. If the binding wins, persist the immutable acceptance context, aggregate, audit evidence, four initial finality records, and one pending `TokenOperationAccepted` outbox row.
4. If the binding loses, compare canonical metadata: return the original aggregate for an exact replay or raise conflict for a different request.
5. Commit before returning the application result and therefore before HTTP 202.

The repository reconstructs the whole aggregate through domain guards rather than handing an opaque blob to callers. Tests cover schema/index shape, replay, conflicting and duplicate two-thread races, forced rollback, optimistic version conflict, microsecond stability, complete histories, participant isolation, and restart read-back.

5.3. Outbox truth

One `PENDING` outbox row proves that acceptance and future-work intent share the local transaction. It does not prove a lease was acquired, a message was published, a consumer deduplicated it, processing began, or an external effect occurred. Exactly-once delivery is not claimed. The row exists so a later worker can implement at-least-once delivery without reopening the request transaction.

Listing 5.1. Illustrative pseudocode: future outbox delivery

```
claim a bounded batch with database-authoritative lease + expiry
for each event:
    publish with stable event identity
    record delivery outcome without changing business truth
on duplicate delivery:
    consumer inbox returns original business effect
on crash or ambiguity:
    recover lease and inquire before any value-moving retry
```

This listing is intentionally not Java or SQL from the pin. It is the Phase 3B pattern that must receive its own migration, ports, worker implementation, and deterministic crash/duplicate tests.

5.4. Which decisions this implements

ADR-001's durable internal truth appears as normalized operation state and histories. ADR-004 keeps the control plane in Java. Implementation ADR 0004 makes the database the concurrency authority and isolates the outer adapter behind the application-owned repository interface. The transaction performs no signing, messaging, RPC, or other external effect.

5.5. Failure timeline and durable interpretation

Persistence review is clearest as a timeline: identify the last transaction that is known to have committed, then derive the only safe next action. A process exception is not itself evidence of rollback, and a caller timeout is not evidence that the server never committed. Stable scope, key digest, and canonical metadata make those unknowns recoverable through replay rather than guesswork.

Table 5.1.. Acceptance failure timeline

Failure point	Durable interpretation	Safe recovery
Before transaction begins	no acceptance evidence exists	retry the same scoped command and key
After binding attempt, before commit	outcome unknown to caller; database resolves commit or rollback	replay the same scope/key/digest
After commit, before HTTP response	operation and pending outbox row exist together	replay returns the original operation
During aggregate read-back	committed state remains authoritative even if representation fails	participant-scoped read/replay; do not create a replacement
Future worker claims event then crashes	acceptance remains true; delivery ownership may expire	recover lease and consumer identity; never rewrite acceptance
External effect response is lost	local attempt may be ambiguous	inquire/observe by stable lineage before authorizing another attempt

Schema evolution must preserve that interpretation across mixed binaries and rollback. Forward-only migrations should add durable concepts with explicit defaults or backfill rules, then allow readers and writers to overlap safely before old fields

are retired. An outbox payload version is a contract with delayed consumers; changing a Java class does not migrate already committed events. Aggregate reconstruction tests should include old rows, current rows, and histories at their boundary cardinalities.

Operational database work is also part of the financial boundary. Backup/restore, replication, failover, clock behavior, connection exhaustion, migration locking, and capacity limits can change availability or recovery posture even when repository unit tests are green. Those controls are absent from the pin and must be demonstrated in the deployment profile rather than inferred from PostgreSQL product capability.

5.6. What remains to be implemented

The schema has no authoritative double-entry ledger, posting sets, reservations, corrections, delivery leases, durable inbox, broker receipts, poison/dead-letter path, execution attempts store beyond domain persistence, independent observation store, reconciliation breaks/cases, repair commands, or production backup/restore/HA controls. Phase 3B must add worker and recovery behavior without weakening the acceptance transaction.

Implementation notes. Treat PostgreSQL integration tests as part of the behavioral contract. An H2 or mock repository cannot prove conflict visibility, constraints, numeric behavior, transaction rollback, or restart reconstruction. Keep external work strictly outside the database transaction.

Continue the thread. Volume I: authoritative ledger, transaction boundaries, outbox/broker/ inbox, reconciliation. Volume II: persistence, messaging, recovery, and operations. ADRs: ADR-001/004 and implementation ADR 0004. Shared modules: `shared/engineering/standards/`, `shared/engineering/observability/`, and `shared/engineering/testing/`. Implementation paths: `adapters/persistence-postgres/` and the application repository port.

6. Wallets, signing, and SignerPort

Why this chapter. Signing is separate authority, not a helper method. The current repository defines a strong request/result contract but deliberately contains no key material or signer implementation. Reading that boundary accurately prevents an interface from being mistaken for custody.

Implementation reference. Signing boundary — digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64; effect-bound SignerPort, SigningRequest, and SigningDecision plus contract tests; application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java;application/src/test/java/io/github/johnwhitton/digitalbanking/application/port/PortContractTest.java; status: **interface only**.

6.1. Why the signing boundary exists

Application authorization answers whether a business action may proceed. Signing authorization answers whether exact native bytes, for one operation/attempt and bounded effect, may use a named key version. On-chain authorization answers whether the network will accept the signer and action. None of these authorities is transitive. An HSM can protect a key while still acting as an arbitrary signing oracle if its caller is not constrained.

Exact pinned excerpt: SignerPort and SigningRequest

digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java · lines 17–42

```
public interface SignerPort {

    SigningDecision sign(SigningRequest request);

    record SigningRequest(
            String signerRequestIdentity,
            OperationId operationId,
            AttemptId attemptId,
            OperationKind kind,
            String chainIdentity,
            String routeVersion,
            AssetUnit unit,
            TokenQuantity quantity,
            String sourceAuthority,
            String opaqueDestination,
            String actionIdentity,
            String nativeConstraintDigest,
            String feeCeiling,
            String allowlistReference,
            byte[] unsignedPayload,
            String unsignedPayloadSha256,
            Instant issuedAt,
            Instant expiresAt,
            String policyVersion,
            EvidenceRef approvalEvidence,
            EvidenceRef simulationEvidence) {
```

The request binds stable signer-request, operation, and attempt identities; operation kind; chain and route; versioned unit and exact quantity; source/destination/action; native constraint digest; fee ceiling; allowlist; exact unsigned bytes and SHA-256; issue/expiry; policy version; approval evidence; and simulation evidence. The full record validates bounded text, digest format, quantity/unit equality, payload digest, time order, nulls, and defensive byte copies. Its toString redacts sensitive decision context.

Exact pinned excerpt: SigningDecision result

digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java · lines 142–169

```
    record SigningDecision(
            Decision decision,
```

```
        String providerRequestIdentity,
        String keyReference,
        byte[] signedPayload,
        String digestReference,
        String reason,
        String signerPolicyVersion,
        EvidenceRef authorizationEvidence) {

    public SigningDecision {
        Objects.requireNonNull(decision, "decision");
        providerRequestIdentity = requireText(
                providerRequestIdentity, "providerRequestIdentity");
        keyReference = requireText(keyReference, "keyReference");
        signedPayload = Objects.requireNonNull(signedPayload, "signedPayload").clone();
        digestReference = requireSha256(digestReference, "digestReference");
        reason = requireText(reason, "reason");
        signerPolicyVersion = requireText(signerPolicyVersion, "signerPolicyVersion");
        Objects.requireNonNull(authorizationEvidence, "authorizationEvidence");
        if (decision == Decision.APPROVED && signedPayload.length == 0) {
            throw new IllegalArgumentException("approved signing decision requires a payload");
        }
        if (decision == Decision.REJECTED && signedPayload.length != 0) {
            throw new IllegalArgumentException(
                    "rejected signing decision cannot contain a signed payload");
        }
    }
```

The public result is named `SigningDecision`, not `SigningResult`. Approved decisions require a signed payload; rejected decisions must contain none. The record also binds provider request identity, key reference, digest, reason, signer policy version, and authorization evidence, while defensive copying and redacted diagnostics remain part of the contract.

One interface limitation is visible in that exact excerpt: both `providerRequestIdentity` and `keyReference` must be nonblank even for a rejected decision. The current contract therefore cannot represent a genuinely pre-provider denial with conditionally absent provider metadata. Production evolution must either distinguish the durable signer-request identity from an optional provider invocation identity or introduce an equally explicit conditional result model; this handbook does not invent a sentinel provider call that never occurred.

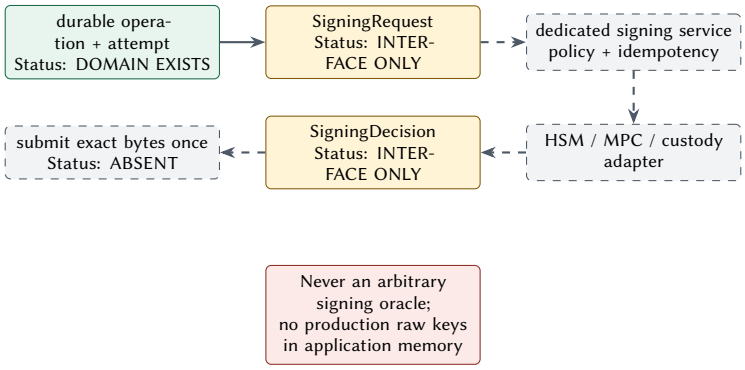

Figure 6.1.. Signing flow and authority boundary. Only the request/result contract exists at the pin.

6.2. Wallet and key identity

No wallet model is implemented beyond source/destination fields in the signing contract. A production-oriented path needs a provider-neutral key registry that maps stable public key identity and version to algorithm, public material, allowed chain and purpose, provider reference, lifecycle state, activation/retirement time, and evidence. Participant, issuer mint, burn, treasury, fee-payer, administration, pause, upgrade, recovery, observer, and reconciler roles must remain distinct unless an explicit production-gap decision combines them.

Signing idempotency also needs durable storage. The durable binding must cover the full canonical authorization request and the resolved immutable key version, including the exact payload—not merely the payload digest. The same request identity and same binding return the original decision; a change to any bound field or key version conflicts. A timeout is ambiguous and must be inquired at the signer/provider before a new request. Rotation crosses registry, policy, provider, on-chain authority, traffic cutover, reconciliation, retention, destruction, and provider-exit evidence.

6.3. Which decisions this implements

ADR-011 defines the wallet model, ADR-012 separates custody/signing authority, and ADR-005 keeps provider/native types out of application signatures. The contract also implements the richer Volume II exact-effect guidance. It does not implement the dedicated signing-service boundary required by `V3R-SIGN-004`; that service is a future adapter-side capability, not code hidden in `SignerPort`.

6.4. A signing ceremony as a durable protocol

The future signing service should treat a request as a protocol record, not a transient RPC call. Before cryptography, it resolves the stable key version and verifies chain, route, action, source/destination, unit, exact quantity, native constraints, fee ceiling, validity window, policy version, approvals, simulation evidence, payload digest, and request identity. Only then may a provider-specific adapter invoke a key. The result and its evidence are committed before a success response leaves the service.

Table 6.1.. Signing protocol checkpoints

Checkpoint	Durable question	Failure posture
Identity	Is this the same signer request, operation, attempt, key version, and payload?	replay exact decision or conflict
Authorization	Do policy and approvals authorize this bounded effect now?	deny without signed payload
Construction	Do exact bytes and digest match approved native constraints?	reject before provider invocation
Invocation	Did the named provider/key produce or refuse a signature?	persist provider request identity and classified outcome
Ambiguity	Was the provider response lost after possible invocation?	inquire by stable request identity; do not resubmit blindly
Verification	Does the signature recover/verify against expected key and payload?	quarantine result and raise evidence-backed failure
Release	Is the signed payload safe to disclose to the one authorized submit path?	never log or return through participant API

Idempotency is effect-bound. After key resolution, the service fingerprints every canonical authorization field, the exact bytes, and the resolved immutable key version. The same signer-request identity and same fingerprint return the recorded decision, including a denial; changing any bound field conflicts. Generating a new request merely to escape an ambiguous outcome defeats the boundary. Rotation similarly uses an explicit cutover: old and new key versions may coexist for verification, but policy determines which version may authorize new work. Historical evidence retains the actual version used.

Local development needs the same protocol with a visibly synthetic key adapter. Its configuration, key format, network access, and artifact packaging should be impossible to activate in a production profile. That guard is more important than algorithmic convenience: an automatic fallback from HSM failure to a local key would erase the separate signing authority the port is designed to preserve.

6.5. What remains to be implemented

There is no dedicated signing service, request store, key registry, wallet registry, local development signer, mock HSM protocol adapter, HSM/MPC/custody integration, cryptographic invocation, audit export, rotation ceremony, disaster recovery, or provider-exit path. The result contract also needs an explicit conditional model for pre-provider rejection metadata. A local signer must be isolated from production configuration and must never become a fallback. Native EVM low-s/recovery and Solana exact-message/signer-order evidence belong to their adapters and tests.

Implementation notes. Implement the service and durable request identity before any local key adapter. Deny unknown action, route, destination, amount, lifetime, policy, or approval by default. Return evidence-backed decisions, never raw provider responses, and never place production private keys in application memory.

Continue the thread. Volume I: key custody and authority boundaries. Volume II: Wallet, Custody, and Signing; EVM/Solana signing; rotation. ADRs: ADR-005/011/012. Shared modules: `shared/engineering/custody/` and `shared/engineering/wallets/`. Implementation paths: `application/src/main/java/.../port/SignerPort.java` and `application/src/test/java/.../port/PortContractTest.java`.

7. Chain adapters

Why this chapter. A language-neutral port protects the core only if native adapters retain their own transaction, lifetime, failure, and finality semantics. This chapter shows the implemented seam and the two deliberately unimplemented native paths.

Implementation reference. Chain boundary — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; capabilities plus separate prepare, submit-once, inquiry, and observation contracts; `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`; `application/src/test/java/io/github/johnwhitton/digitalbanking/application/port/PortContractTest.java`; status: **interface only**.

Exact pinned excerpt: ChainPort lifecycle

`digital-banking@a58ca0ad6834ab53d80a0431e97d12c25b4d6d64` · `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java` · lines 14–30

```
public interface ChainPort {

    ChainCapabilities capabilities(String routeVersion);

    PreparedAttempt prepare(TokenOperation operation, OperationAttempt attempt);

    SubmissionResult submitOnce(SignedAttempt signedAttempt);

    InquiryResult inquire(AttemptIdentity attemptIdentity);

    Observation observe(ObservationRequest request);

    record ChainCapabilities(
            boolean supportsMint,
            boolean supportsBurn,
            boolean supportsIndependentInquiry) {
    }
```

7.1. Why the port exists

The port gives the application a stable sequence without exposing Web3j, JSON-RPC, Solana SDK, transaction, receipt, account, blockhash, slot, or commitment types. It expresses capability discovery; deterministic preparation; submission of exact signed bytes once; inquiry by stable operation/attempt/native identity; and materially independent observation. Results classify accepted, definitively rejected, and ambiguous submission plus retry safety. No method is named "settle."

Invariant. Submission acknowledgement is evidence only. A timeout or lost response is ambiguous. The application inquires and observes by stable identity before authorizing any new value-moving attempt.

7.2. Planned EVM execution

Implementation ADR 0002 accepts Solidity and Foundry for required on-chain enforcement, Anvil for local execution, and adapter-local Web3j for Java integration. It authorizes no public network or funded account (Foundry Contributors 2026; Ethereum Foundation 2026). At the pin there is no EVM adapter, no contract, no Foundry configuration or version pin, no Web3j dependency, no nonce store, and no RPC provider.

The eventual adapter must bind chain ID, allowlisted contract/function, exact calldata, value, nonce, gas/fee bounds, expiry, policy, approvals, and simulation evidence before signing. It must own nonce reservation, one signed-byte submission, replacement lineage, transaction identity, receipt/log decoding, confirmation policy, reorg/canonicality, independent read comparison, finality evidence, and reconciliation. A replacement is a related attempt, not permission to erase the predecessor.

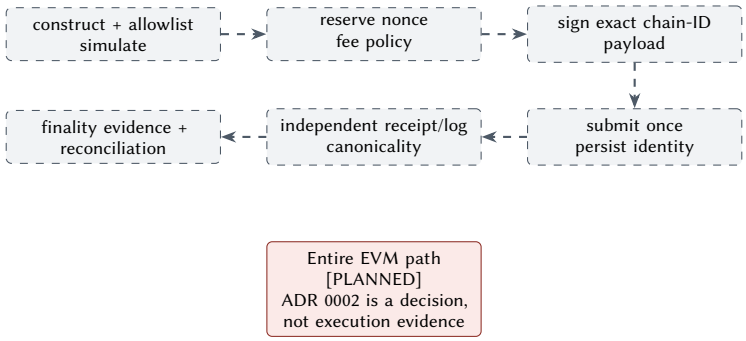

Figure 7.1.. Planned EVM execution sequence preserving nonce, replacement, receipt, and reorg semantics.

7.3. Planned Solana execution

Implementation ADR 0003 selects native SVM semantics and the classic SPL Token Program for ordinary token operations, rejects a speculative custom program, and requires a bounded Java-client evaluation before accepting a dependency (Solana Foundation 2026b; Solana Foundation 2026a). At the pin there is no Solana adapter, SDK, local validator, Rust/Anchor workspace, custom program, token configuration, account builder, blockhash service, or RPC provider.

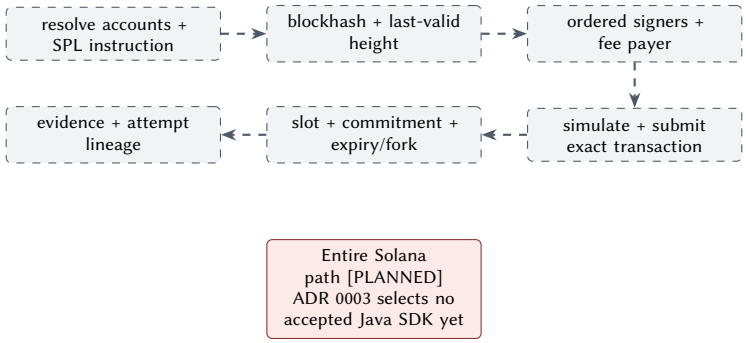

Figure 7.2.. Planned Solana execution sequence. A new blockhash means reconstruction and new attempt lineage, not blind replay.

The eventual adapter must preserve account metas, instruction order, program identity, mint and token-account authorities, fee payer, signer ordering, exact message bytes, recent blockhash and last-valid block height, compute and priority-fee bounds, simulation, submission identity, slot/commitment, expiry/drop/fork handling, independent observation, and reconciliation. Reusing the same signed transaction within its lifetime differs from reconstructing with a new blockhash; reconstruction changes the message and signature and therefore requires linked attempt lineage.

Table 7.1.. Native semantics that must not be flattened

Concern	EVM owner	Solana owner
Replay/lifetime	account nonce and replacement policy	blockhash, last-valid height, durable nonce choice
Payload	typed transaction, chain ID, calldata	ordered message, accounts, instructions, lookup tables
Authority	recovered ECDSA sender and contract roles	fee payer, ordered Ed25519 signers, program/account authorities
Observation	receipt, logs, block canonicality, confirmations	signature status, logs, slot, commitment, fork/expiry
Rebuild	fee-bumped related replacement	new message/signature and linked attempt

7.4. Which decisions this implements

The current code implements ADR-005 and ADR-013 only at the interface boundary. ADR-008 requires first-class Ethereum/Base/Solana evaluation, while implementation ADRs 0002 and 0003 establish future native tool directions. None of these decisions is runtime evidence. EVM and Solana adapters must never depend on each other.

7.5. The common-to-native handoff

The application should hand an adapter a bounded business effect and receive evidence, not ask it to improvise a transaction. Preparation resolves configured native identity, constructs the canonical payload, simulates it where supported, and returns unsigned bytes plus a digest and lifetime constraints. Signing authorizes exactly that artifact. Submission accepts the signed bytes once and records the adapter's stable native identity. Inquiry and observation then operate on that lineage without reconstructing a new effect.

Table 7.2.. Port phase versus native ownership

Phase	Common contract	Adapter-owned detail	Durable evidence
Capabilities	named route and supported operation	network/program/contract and feature limits	versioned capability/configuration identity
Prepare	exact effect, constraints, stable attempt	nonce or blockhash, fee model, accounts/calldata, simulation	payload bytes/digest, lifetime, simulation reference
Sign	exact bytes and effect-bound request	ECDSA or Ed25519 rules and signer order	key version, signed digest, authorization reference
Submit once	same signed bytes and attempt	RPC method, native identity, transport result	accepted, rejected, or ambiguous classification
Inquire	stable business/native lineage	transaction/signature status semantics	provider response reference and retry-safety decision
Observe	same lineage through independent path	canonical block/slot, logs/accounts, confirmations/commitment	observation source, position, assessment, provenance
Reconcile	compare internal, submit, observer, ledger truths	reorg/fork/replacement/expiry interpretation	break, owner, action, and repair authorization

Submission classification must be conservative. A deterministic local rejection before the node accepts bytes may permit correction under policy. A node acknowledgement is not finality. A timeout after bytes leave the process is ambiguous even if the client library throws a retryable exception. The adapter reports those facts; only the application workflow can decide whether inquiry is complete and a related attempt may be authorized.

Independent observation must also be materially independent in credentials, endpoint, code path, and evidence. Calling the same provider twice through two methods does not protect against provider omission or a shared parsing defect. The common finality model can store assessments from both chains, but the adapter owns the native facts used to produce them and the reconciler owns disagreement handling.

7.6. What remains to be implemented

Everything after the port: adapter modules, dependency/version pins, local native environments, construction, simulation, signer integration, persistent native identity and lifetime stores, submission, inquiry, independent observation, finality policy, reconciliation, failure injection, and security review. The sequence remains worker and signing controls first, one EVM vertical slice second, and Solana validation against the same business contract third.

Implementation notes. Keep shared application contracts small and native adapter models rich. Prove deterministic bytes and identities before submission, and test ambiguity, expiry, replacement/fork, and observer disagreement locally before considering any provider integration.

Continue the thread. Volume I: chain abstraction, failure/finality, no-bridge default. Volume II: EVM Engineering, Solana Engineering, Submission and Observation. ADRs: ADR-005/008/013/015 and implementation ADR 0002/0003. Shared modules: `shared/engineering/sdk/`, `shared/engineering/smart-contracts/`, and `shared/engineering/chains/`. Implementation path today: `application/src/main/java/.../port/ChainPort.java`; future paths are named in the implementation ADRs.

8. Testing

Why this chapter. The repository's implemented scope is best defined by its executable tests. A truthful test chapter names what each layer proves, what environment it uses, and which required failure and native layers do not yet exist.

Implementation reference. Committed test architecture — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; pure domain/application tests, port contract tests, real PostgreSQL persistence tests, Spring/API/security tests, and OpenAPI conformance; `domain/src/test;application/src/test;adapters/persistence-postgres/src/test;control-plane/src/test`; status: **partially implemented**.

8.1. Why the test architecture exists

Inner tests provide fast, deterministic proof of exactness and lifecycle without frameworks. Persistence tests use the actual selected database because PostgreSQL conflict visibility, constraints, numeric behavior, transaction rollback, and restart reconstruction are part of the design. Control-plane tests add security, representation, commit-before-202, problem mapping, and OpenAPI behavior. This is evidence layering, not a coverage-number exercise (Testcontainers 2026).

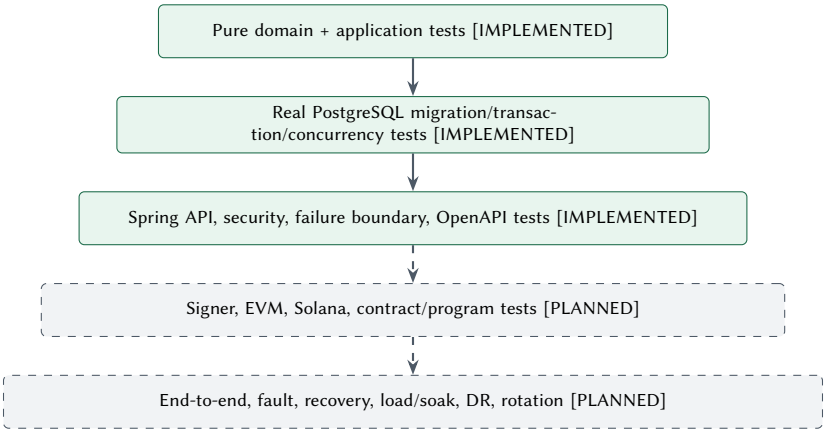

Figure 8.1.. Testing architecture. The pyramid widens into explicit absent layers rather than implying coverage.

8.2. Current executable evidence

The pinned README records a 302-test reactor gate and 269 domain/application tests. The Phase 2C release gate freshly ran `clean verify` from an exact Git archive of the same commit with JDK 25.0.2, Maven Wrapper 3.3.4/Maven 3.9.16, Docker Desktop 28.5.1, Testcontainers 2.0.5, and PostgreSQL 17.10; it observed 307 passing tests, including 270 in domain/application. The retained `volume3/evidence/implementation-clean-verify.txt` binds that observation to the exact Git-archive SHA-256, command, UTC interval, per-module totals, environment, and Surefire report-set digest. The five-test documentation lag is itself a reason to treat counts as inventory rather than behavior. Assertions and environment matter more than totals. The committed test tree establishes the following bounded evidence:

Table 8.1.. Implemented test layers and limits

Layer	Named evidence	What it proves / does not prove
Domain	`TokenQuantityTest`, `TokenOperationLifecycleTest`, `TokenOperationEvidenceTest`	exact values, transitions, attempts, evidence, and finalities; no persistence or external effects
Application	canonicalizer, application service, and service tests	canonical command, replay/conflict, orchestration, deterministic clocks/IDs; no Spring or provider
Port contract	`PortContractTest`	signing-field binding and separate chain seams; no cryptography or RPC
PostgreSQL	`PostgresOperationRepositoryTest`	migration, mapping, constraints, transaction, concurrency, rollback, restart, outbox acceptance; no HA or delivery
Control plane	application, readiness, API integration/failure, response, and OpenAPI tests	composition, security, safe representations/errors, HTTP semantics; fixture identity only

8.3. Failure paths already covered

Current tests include invalid quantities and identifiers, forbidden lifecycle transitions, attempt ordering, finality regression, same-key replay, same-key conflict, participant/kind isolation, two-thread duplicate/conflicting acceptance, forced transaction rollback, optimistic conflict, restart read-back, missing/incorrect authority, participant non-disclosure, malformed input, unsupported media, dependency failure, unexpected invariant failure redaction, and OpenAPI drift.

These failures stop at durable acceptance. No test can prove an absent worker, signer, chain adapter, native contract/program, observer, reconciler, or production runtime.

8.4. Required future test slices

1. Worker leasing, duplicate delivery, crash-before/after transition, recovery scan, bounded retry, poison/manual review, and restart.
2. Signer exact-byte, policy denial, same-request replay/conflict, provider timeout/ inquiry, concurrency, redaction, rotation, and no-key-leakage.
3. EVM deterministic encoding, nonce races, replacement, lost response, receipt/log, reorg/canonicality, contract authorization/pause, and independent observation on Anvil.
4. Solana accounts/instructions, signer ordering, fee payer, blockhash expiry, same-signed replay, reconstruction lineage, commitment/fork, account contention, and independent observation on a local validator.
5. End-to-end duplicate, outage, ambiguity, disagreement, reconciliation break, controlled repair, data restore, load/soak, service/region loss, and provider exit.

Every failure test should state safety preconditions, injected fault, expected durable state/evidence, permitted recovery, prohibited shortcut, and reconciliation proof. A wall-clock sleep is not a concurrency oracle; use barriers, latches, explicit timeouts, and outcome sets.

8.5. Which decisions this implements

Tests enforce ADR-001/002 truth and finality, ADR-004/005 module ownership, and implementation ADR 0004's database behavior. They also implement the lower layers of the Volume II test strategy. The complete `V3R-TEST-001..005` catalog remains open because native, end-to-end, recovery, load, and rotation layers are absent.

8.6. From a green test to an admissible claim

A test result becomes implementation evidence only when its subject, environment, assertion, and limitation are named. “302 tests pass” establishes that Maven observed 302 passing cases in one recorded run; it does not establish which transaction race, database version, security boundary, or absent adapter was covered. The evidence register therefore links claims to focused tests and preserves the full reactor result as a separate build gate.

Table 8.2.. Evidence ladder for an implementation claim

Level	Acceptable claim	Still not established
Source inspection	interface, guard, SQL, mapping, or configuration exists at the pin	runtime path or failure behavior
Focused pure test	deterministic invariant or orchestration outcome	adapter semantics or durable restart
Real-adapter test	selected database/provider behavior under named conditions	production topology, scale, HA, or governance
Module integration	composed boundary, mapping, security, and transaction behavior	native/end-to-end effect if dependency is absent
System failure test	durable recovery across injected fault and restart	untested fault combinations or organization readiness
Operational exercise	observed load, restore, failover, rotation, or exit result	future capacity or universal production safety

Each future test report should record the implementation commit; tool, runtime, image, contract/program, and configuration versions; deterministic fixture identities; test selection; start/end time; result artifacts; and known exclusions. A failure-injection case also records the fault point, last known durable state, expected recovery owner, and prohibited duplicate effect. This makes a rerun comparable instead of merely green.

Negative tests deserve first-class review because they define authority. A missing principal must not become a fixture principal. A denied signer request must contain no payload. An ambiguous chain response must not become a new submission. A reconciliation disagreement must not be overwritten by the preferred observer. These assertions are often more important than the happy path because they prove the shortcuts the system refuses under pressure.

8.7. What remains to be implemented

Add only the tests owned by each new slice, together with its executable environment. There are no signer-adapter, EVM, Solana, contract/program, end-to-end, fault-injection, load/soak, disaster-recovery, credential-rotation, or provider-exit suites today. Test reports must name exact tool versions and must never be presented as production certification.

Implementation notes. Run focused pure tests first, real adapter tests second, and the full reactor gate last. Preserve deterministic identities and time. Review assertions and source together; a green test count without environment and claim boundaries is not implementation evidence.

Continue the thread. Volume I: invariant and failure requirements. Volume II: Testing and Verification plus failure catalogs. ADRs: ADR-001/002/004/005 and implementation ADR 0004. Shared module: `shared/engineering/testing/`. Implementation paths: every module’s `src/test/java/`, root `pom.xml`, and the machine-readable `volume3/evidence/test-catalog.csv`.

9. Running locally

Why this chapter. A runnable command is useful only when it states which dependencies exist and what success means. Today the repository can build and run a Spring process against PostgreSQL; it cannot execute the settlement architecture end-to-end.

Implementation reference. Local application boundary — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; Maven wrapper build, packaged Spring application, PostgreSQL datasource, health/readiness, and static OpenAPI resource; `mvnw;pom.xml;control-plane;README.md`; status: **partially implemented**.

9.1. Why a local boundary exists

Local execution gives contributors a repeatable way to compile all modules, run the implemented test layers, migrate a disposable PostgreSQL database, start the control plane, and inspect health plus the contract. It is not a demo of minting, burning, signing, chain submission, observation, reconciliation, or settlement.

The pinned build requires JDK 25. The repository wrapper resolves Apache Maven 3.9.16, and Spring Boot is 4.0.6. Persistence integration tests start the pinned PostgreSQL container automatically, so Docker is needed for the clean verification gate. First dependency and image resolution may require network access; later clean builds use the resolved caches.

Listing 9.1. Current build commands from the pinned repository

```
JAVA_HOME=/opt/homebrew/opt/openjdk ./mvnw --version
JAVA_HOME=/opt/homebrew/opt/openjdk ./mvnw clean verify
```

These workstation commands are evidence of the documented bootstrap environment, not a portable promise that Homebrew lives at the same path elsewhere. On another system set `JAVA_HOME` to its JDK 25 installation.

9.2. Start the current application

Provide a private/local PostgreSQL 17 database. Flyway validates and migrates it at startup; there is no in-memory durable fallback.

Listing 9.2. Current local application launch pattern

```
SPRING_DATASOURCE_URL="${DB_URL}" \
SPRING_DATASOURCE_USERNAME="${DB_USERNAME}" \
SPRING_DATASOURCE_PASSWORD="${DB_PASSWORD}" \
JAVA_HOME="${JDK25_HOME}" "${JDK25_HOME}/bin/java" \
  -jar control-plane/target/digital-banking-control-plane-0.1.0-SNAPSHOT.jar
```

In a second terminal, inspect readiness and the design-first contract:

Listing 9.3. Current safe inspection endpoints

```
curl --fail --silent http://localhost:8080/actuator/health/readiness
curl --fail --silent http://localhost:8080/openapi/token-operations-v1.yaml
```

Readiness `UP` means the application considers its configured health components ready. It does not mean business endpoints can accept a request or that any external dependency exists. The default repository config contains no identity adapter and an empty asset catalog, so secured business resources deny unauthenticated calls and normal acceptance lacks a configured unit. Tests inject fixture principals and catalogs.

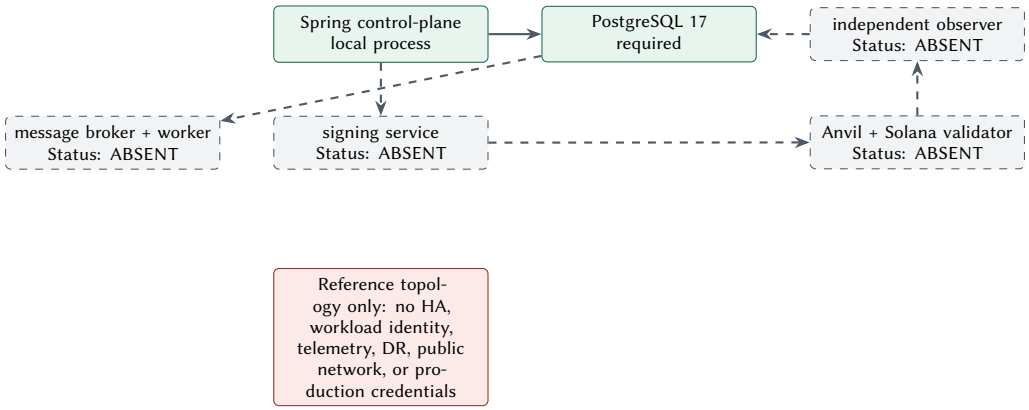

Figure 9.1.. Current local topology and planned dependencies. Only the Spring process and PostgreSQL relationship are executable.

9.3. What a complete local reference runtime needs

A future `V3R-RUNTIME-001` environment must pin and orchestrate the control plane, PostgreSQL, broker/worker, synthetic identity/policy/bank/issuer dependencies, dedicated signing service and local-only signer, Anvil EVM path, Solana local validator path, materially independent observer, reconciliation service, telemetry, deterministic fixtures, and a failure-control harness. It must make secret-free local defaults and production lockouts visible at the point of execution.

Containers and sample infrastructure are illustrative deployment artifacts. They need health ordering, persistence volumes, identity, network policy, resource bounds, configuration, rollback, diagnostics, and cleanup, while explicitly omitting production availability, recovery, scale, and governance. There is currently no Compose, Kubernetes, Terraform, CI/CD deployment, or telemetry stack.

9.4. Interpreting local signals

Local diagnosis should start from the narrowest signal and avoid upgrading it. A successful compile proves source and dependency compatibility for the selected toolchain. A green pure test proves its assertions without outer systems. A green persistence test adds the pinned PostgreSQL behavior. A running process and readiness response prove only the configured application boundary. A 401 on a business resource is expected with the default identity boundary; weakening security to make the request return 202 would make the local environment less representative.

Table 9.1.. Local signal interpretation guide

Signal	What it establishes	Next diagnostic boundary
Wrapper reports pinned Maven/JDK	selected build toolchain is active	dependency resolution and reactor compilation
Pure module tests pass	domain/application assertions pass	real persistence adapter tests
PostgreSQL tests pass	selected schema, SQL, transactions, and restart cases pass	Spring composition and transport
Application starts	configuration, migration, and composition reached running state	readiness components and logs
Readiness is UP	configured health contributors report ready	identity and asset configuration for business use
Business request returns 401	deny-by-default boundary is active	install a deliberate local identity adapter, not a bypass
OpenAPI is retrievable	static contract resource is served	contract/behavior conformance tests

A complete future runtime should expose a stable verification script rather than a sequence of undocumented terminal gestures. That script should assert tool/image versions, start services in dependency order, wait on explicit conditions, install only synthetic fixtures, execute success and controlled-failure scenarios, capture evidence, and clean up without

leaving credentials or funded state. A failure must identify the service and last satisfied condition so contributors do not compensate by adding sleeps or retrying value-moving calls.

The local signer and native networks need hard production lockouts. Configuration should reject public chain identities, externally funded accounts, production issuer keys, and production identity providers in the local profile. Conversely, production packaging should exclude synthetic keys and fixture authentication entirely. The two modes should not be separated by a single permissive boolean.

9.5. Which decisions this implements

Implementation ADR 0001 supplies the reproducible Java wrapper and module build. Implementation ADR 0004 supplies PostgreSQL startup/migration behavior. Series ADR-013 requires future submit and observer paths to remain independent. The current two-process shape is only a bounded developer environment, not the deployment topology in Volume I.

9.6. What remains to be implemented

There is no execution worker, message broker, durable scheduler, identity adapter, signing service, local key adapter, EVM node/adapter, Solana validator/adapter, independent observer, reconciliation/case service, seeded business fixture, end-to-end flow, failure harness, telemetry, or production deployment example. Until these exist, "run locally" means build/test and inspect durable acceptance infrastructure only.

Implementation notes. Use synthetic local data and obvious placeholders. Never add a public-network endpoint, funded address, production credential, default user, or raw key to make a demo easier. Keep every service version pinned and every success statement bounded to its observed health or test.

Continue the thread. Volume I: trust/deployment topology and staged delivery. Volume II: Infrastructure and Operations, local-chain testing, observability. ADRs: ADR-003/013 and implementation ADR 0001/0004. Shared modules: `shared/engineering/deployment/`, `shared/engineering/infrastructure/`, and `shared/engineering/observability/`. Implementation paths: root `README.md`, `mvnw`, `control-plane/`, and `application.yaml`.

10. Production evolution

Why this chapter. The current code is a foundation, not a miniature production system. Production evolution must therefore close authority, financial-state, recovery, native, security, and operational gaps in dependency order rather than adding deployment polish around durable acceptance.

Implementation reference. Living delivery plan — `digital-banking@ a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`; implemented Phase 3A durable acceptance and evidence-gated future slices; `docs/DESIGN.md;docs/IMPLEMENTATION.md;docs/adr;docs/plans/active`; status: **partially implemented**.

10.1. Why evolution is gated

External value movement multiplies the number of truths and failure modes. Before a signature or chain call, the system needs durable work identity, retry ownership, and recovery. Before production, it also needs authoritative ledger and reconciliation, independent observation, qualified signing authority, identity/policy/compliance, security, service levels, recovery, support, and organization governance. These are release gates, not backlog decorations.

The full future Volume III implementation criterion `V3R-ACC-002` is not met at the pinned commit. This handbook does not reinterpret that criterion. Instead, the production-gap register gives every current module a classification, omitted controls, consequence, replacement contract, owner, and gate.

10.2. Dependency-ordered path

Table 10.1.. Evidence-gated implementation sequence

Gate	Slice	Exit evidence before proceeding
1	Worker and outbox recovery	database-authoritative leases, durable timers, at-least-once publication, inbox/deduplication, restart/crash/duplicate tests, manual review route; no external value effect
2	Payment/transfer and ledger	parent identity, child effect lineage, balanced postings/reservations/corrections, bank-port inquiry, compensation and reconciliation invariants
3	Identity, policy, and approval	approved participant/workload identity, tenant isolation, policy/compliance evaluation, approval evidence, dual control where required, and state-transition tests proving policy precedes `AUTHORIZED` and attempt creation
4	Signing authority	dedicated service, durable request idempotency, key registry, full-request-plus-key-version binding, conditional pre-provider rejection metadata, local-only signer, mock provider failures, policy denial, rotation and provider-exit contracts
5	EVM vertical slice	pinned Foundry/Web3j, bounded contract, deterministic bytes, nonce/replacement store, local Anvil submission, independent observation, reorg and reconciliation tests
6	Solana vertical slice	accepted Java client after gate, native SPL instructions, exact message/signers, lifetime/reconstruction lineage, local-validator observation, fork/expiry and reconciliation tests
7	Integrated local system	synthetic identity/policy/banks, broker, both native paths, observer/reconciler, stable fixtures, end-to-end success and controlled failure catalog

Gate	Slice	Exit evidence before proceeding
8	Operational and security hardening	telemetry-versus-evidence boundary, SLOs, capacity, backup/restore, DR, supply chain/SBOM, threat/privacy review, runbooks and clean-room reproduction
9	Organization production decision	approved profile, legal/compliance/custody and provider selection, region/scale/RTO/RPO, independent audits, risk acceptance and pilot exit criteria

Each slice must update the implementation evidence register and the retrofit register. A discovery that changes Volume I/II guidance is reviewed there before publication text changes; code does not silently become architecture authority.

10.3. Production boundary by concern

Table 10.2.. Current foundation versus production gate

Concern	Current evidence	Production gate
Financial truth	exact quantity, operation state, four-finality records	authoritative ledger, authority-backed finality, reconciliation/cases/repair
Identity/authorization	deny-by-default boundary and fixture principals	approved identity, tenant isolation, policy/compliance, dual control
Signing	effect-bound interface only	dedicated service, qualified providers, key lifecycle, DR and exit
Chains	prepare/submit/inquire/observe interface only	audited native construction, local then provider tests, independent observation
Reliability	atomic local acceptance and integration tests	workers, retries, ambiguity recovery, SLO/capacity, backup/restore, region strategy
Deployment	manual Spring plus PostgreSQL pattern	workload identity, secrets, network, HA, telemetry, CI/CD, rollback, governance

Production boundary. A passing local build, test suite, contract audit, HSM integration, chain confirmation, or pilot does not by itself establish production readiness. Production is an organization decision supported by complete evidence across financial, legal, security, operational, provider, and recovery authorities.

10.4. Which decisions this implements

The sequence preserves ADR-006's bounded pilot, ADR-012's independent signing authority, ADR-013's independent observation, ADR-014's build-versus-buy review, and ADR-015's no-bridge default. Implementation ADRs 0002–0004 define current tool/data directions without pre-authorizing a vendor, public network, or production topology.

10.5. Change control for implementation evidence

Each implementation advance changes two products: executable behavior and the evidence boundary readers rely on. A pull request for a new slice should therefore name the affected operation states, persistence records, public contracts, authorities, failure classes, telemetry, tests, ADRs, and production gaps. Reviewers can then distinguish a local refactor from a change to financial or recovery semantics.

Table 10.3.. Evidence refresh triggered by implementation change

Change	Required evidence refresh	Release question
Public/domain contract	exact excerpt, digest, focused tests, compatibility decision	can old callers and durable rows retain their meaning?
Workflow transition	state diagram, transaction boundary, recovery and negative tests	what is durable before and after every effect?
Schema/event	migration, reconstruction, mixed-version, rollback, archive replay	can delayed data and consumers still be interpreted?
Signer/provider	key/request lineage, failure inquiry, rotation and exit evidence	can ambiguity or provider loss cause duplicate authority?
Native chain	deterministic payload, lifetime, submit/inquire/observe and fork/reorg tests	are native facts preserved without leaking inward?
Identity/policy	threat model, authority matrix, denial/disclosure tests, audit evidence	can one principal cross participant or operator scope?
Deployment	configuration schema, SBOM, restore/failover/load evidence, runbooks	which environment and failure envelope was actually proved?

The production dossier grows cumulatively. It includes the implementation commit and reproducible build; artifact provenance and dependency inventory; migration and data recovery evidence; identity, policy, custody, and key ceremonies; native contract or program audits; observer and reconciliation results; capacity and SLO measurements; backup/restore and disaster exercises; privacy, compliance, and legal decisions; operating runbooks; provider exit tests; known gaps; and signed risk acceptance. No single engineering team can self-approve all of those authorities.

Rollback is also a business workflow. Code rollback cannot erase a committed schema, outbox event, signature, submission, or ledger posting. Every slice must define whether rollback means disabling new work, routing to manual review, deploying a compatible old reader, compensating a business effect, or migrating forward. The evidence register keeps that posture tied to the exact commit instead of a floating “current” system.

10.6. What remains to be implemented

Every gate after durable acceptance. The implementation repository should advance one bounded vertical slice at a time, starting with worker/recovery. Volume III should then refresh to a new full commit only after direct source/test review, excerpt regeneration, production-gap reclassification, cross-volume/ADR review, and the complete publication release gate.

Implementation notes. Do not start with Kubernetes, multi-region topology, or two chain adapters. First make durable work recovery correct, then establish the financial and signing authorities, prove one native path, and use the second chain to test whether the common contract remained honest.

Continue the thread. Volume I: roadmap, pilot, risk, ledger, finality, and production boundaries. Volume II: implementation roadmap, infrastructure/operations, security, performance, and economics. ADRs: ADR-006/012/013/014/015 and implementation ADRs 0002–0004. Shared modules: all `shared/engineering/` areas. Implementation paths: `docs/IMPLEMENTATION.md`, active plans, ADR index, and the machine-readable production-gap register in this release.

A. Exact contract provenance

The excerpts printed in Chapters 2–7 are short contiguous fragments from the exact commit. They are stored separately so a verifier can compare bytes with committed Git objects; this avoids both hand-normalization and a substantial second copy of the source. The canonical machine-readable record is `evidence/excerpt-manifest.csv`.

Table A.1.. Exact pinned excerpt register

ID	Contract/status	Repository path and lines	SHA-256 prefix
V3-EX-001	Domain identifier / implemented	`domain/src/main/java/io/github/johnwhitton/digitalbanking/domain/operation/OperationId.java`, 7–36	`9bb5ba392f8e`
V3-EX-002	Repository interface / implemented	`application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/OperationRepository.java`, 14–32	`d3ced736c04d`
V3-EX-003	SignerPort / interface only	`application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java`, 17–42	`53a46d9b34c5`
V3-EX-004	SigningDecision / interface only	same file, 142–169	`bcf456c51a26`
V3-EX-005	ChainPort / interface only	`application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`, 14–30	`bfb6b8226351`
V3-EX-006	Controller / partially implemented	`control-plane/src/main/java/io/github/johnwhitton/digitalbanking/controlplane/api/TokenOperationController.java`, 39–60	`a0cd72cc1b2d`
V3-EX-007	OpenAPI / partially implemented	`control-plane/src/main/resources/static/openapi/token-operations-v1.yaml`, 9–38	`f486428a3936`

All seven rows pin `a58ca0ad6834ab53d80a0431e97d12c25b4d6d64`. The full SHA-256 values, owning tests, and decision references are in the CSV. The release command below fails for a different byte, path, line range, digest, or unreachable commit:

```
make implementation-verify IMPLEMENTATION_REPO=/path/to/digital-banking
```

Exactness does not upgrade status. The signer and chain excerpts remain interface only even though the source matches perfectly. The controller and OpenAPI remain partially implemented because their durable acceptance surface has no production identity or execution path.

A.1. Claim-to-test discipline

An excerpt proves the public shape shown. Its named test proves only the assertions it executes. Runtime behavior claims therefore cite source plus a focused test or a recorded build gate; absence claims cite the committed tree and current-state documents. A future commit reopens every path, line, digest, status, limitation, and cross-volume statement.

B. Implementation evidence and conformance

B.1. Capability summary

Table B.1.. Pinned implementation status

Capability	Status	Limitation
Maven reactor/dependency direction	implemented	four current modules only
Exact quantity and operation IDs	implemented	not a ledger or payment aggregate
Token-operation lifecycle/finality records	partially implemented	no owning worker, observer, reconciliation, or authority assessment
Application acceptance/idempotency	implemented	no external execution orchestration
PostgreSQL durable acceptance	implemented	no ledger, delivery, inbox, or repair
Spring/OpenAPI business surface	partially implemented	default identity absent; no execution surface
Pending acceptance outbox	partially implemented	no claim/lease/publish/consume/recover
SignerPort / SigningDecision	interface only	no signing service, key registry, or provider
ChainPort	interface only	no EVM or Solana implementation
EVM and Solana paths	planned	ADR decisions only
Core/PostgreSQL/API tests	implemented	native/system/failure/recovery layers absent
Local runtime	planned	manual Spring plus PostgreSQL is the only current shape
Production keys/public networks	intentionally excluded	separate authorization required
Full `V3R-ACC-002` acceptance	unresolved	required complete system does not exist

The full record is `evidence/implementation-status.csv`; it includes exact commit, source, tests, implemented decision, and remaining work for every row.

B.2. Specification relationship

The current implementation illustrates parts of `V3R-DOM-001/002`, the public signer and chain seams, and a subset of the evidence/test requirements. It does not meet the release failure/acceptance conditions that require a ledger, worker/messaging, first-class EVM and Solana paths, signer implementation, observation, reconciliation, local multi-service runtime, and complete failure catalog. The publication is therefore a transparent current-state companion, not a conformance certificate.

B.3. Production-gap classes

The machine-readable `evidence/production-gap-register.csv` applies four classifications from the Volume III RFC:

- **Production-oriented pattern**: exact identity, lifecycle, idempotency, explicit persistence, port boundaries, and focused invariant tests.
- **Illustrative integration**: the current Spring API and any future local signer/mock dependencies, always with a replacement contract.
- **Illustrative deployment**: a future local topology with no production HA, security, scale, recovery, or service-level claim.

- **Deferred organization work**: provider/custody/identity/compliance, regions, volumes, RTO/RPO, legal finality, procurement, audit, and production approval.

No test result promotes one class to another. Production recommendation requires the named owner and gate in the register plus the organization-specific governance that this generic repository intentionally cannot supply.

C. Implementation glossary

Term	Meaning at the pinned implementation
Acceptance	One PostgreSQL transaction durably binds scoped idempotency, canonical command, operation state/histories/finalities, evidence, and a pending outbox row. It is not execution.
Attempt	One authorized effort toward an external effect, with stable identity and lineage; no executable native attempt exists yet.
Business truth	Durable internal operation, policy, ledger, finality, and reconciliation state. The current implementation contains only a bounded subset.
ChainPort	Framework-free interface separating capability, preparation, submission, inquiry, and observation. It has no adapter at the pin.
Exact quantity	Versioned asset/unit plus bounded atomic-unit representation parsed from canonical decimal text; not a ledger balance.
Four finalities	Blockchain, legal settlement, customer-visible, and accounting assessments retained independently. Current records have no assessment service.
Idempotency scope	Tenant, participant, resource, operation kind, opaque-key digest, and canonical command metadata used to return original acceptance or conflict.
Independent observation	Materially separate evidence path from submission. It is a port/design responsibility only at the pin.
OperationId	Server-issued canonical UUID business identity that survives retries and does not collapse into a native transaction identity.
Outbox	Pending event intent committed with acceptance. No worker or publication is implemented.
SignerPort	Effect-bound signing request/result interface. The result type is `SigningDecision`; no signer implementation or key material exists.
Token operation	Current mint/burn aggregate with exact quantity, state/version, attempts, evidence, and finality histories; not a full payment or transfer.

The series-wide canonical terminology remains in `specifications/GLOSSARY.md`. This appendix only clarifies how those terms map to the evidence commit.

Annotated implementation references

Ethereum Foundation (2026). *Ethereum Transactions*. Native transaction context for the planned adapter; not evidence of an implemented EVM path. URL: https://ethereum.org/developers/docs/transactions/ (visited on 2026-07-17).

Foundry Contributors (2026). *Foundry Documentation*. Accepted future EVM toolchain direction; no Foundry source or runtime exists at the evidence commit. URL: https://getfoundry.sh/ (visited on 2026-07-17).

John Whitton (2026). *Digital Banking Reference Implementation*. Primary implementation artifact; every claim in this volume is further bounded by repository-relative source and test paths. URL: https://github.com/johnwhitton/digital-banking/tree/a58ca0ad6834ab53d80a0431e97d12c25b4d6d64 (visited on 2026-07-17).

OpenAPI Initiative (2021). *OpenAPI Specification 3.1.0*. Contract format referenced by the pinned design-first API document. URL: https://spec.openapis.org/oas/v3.1.0.html (visited on 2026-07-17).

PostgreSQL Global Development Group (2026). *PostgreSQL 17 Documentation*. Database semantics reference; implementation ADR 0004 and the pinned migration define local behavior. URL: https://www.postgresql.org/docs/17/ (visited on 2026-07-17).

Redgate (2026). *Flyway Documentation*. Migration-tool reference; no claim beyond the version resolved by the pinned Maven build. URL: https://documentation.red-gate.com/flyway (visited on 2026-07-17).

Solana Foundation (2026a). *Tokens on Solana*. Supports ADR 0003's decision to prefer the existing token program before a custom program. URL: https://solana.com/docs/tokens (visited on 2026-07-17).

– (2026b). *Transactions and Instructions*. Native Solana model for planned work; no SDK or validator is present at the evidence commit. URL: https://solana.com/docs/core/transactions (visited on 2026-07-17).

Spring (2026). *Spring Boot Reference Documentation*. Framework reference only; the pinned POM and source establish the implementation's exact use. URL: https://docs.spring.io/spring-boot/index.html (visited on 2026-07-17).

Testcontainers (2026). *Testcontainers for Java*. Integration-test tooling context; the pinned POM and tests establish actual usage. URL: https://java.testcontainers.org/ (visited on 2026-07-17).